

Microservices Demo Cours Del'architecture Client Serveur

Description

Ce projet est une démonstration académique d'une **architecture microservices** développée avec **Django REST Framework (DRF)**.

L'objectif est de montrer comment une application peut être divisée en plusieurs **services indépendants**, communiquant entre eux à travers des API **HTTP REST**.

Le système implémente une plateforme de vote permettant aux utilisateurs de :

1. créer un compte ;
2. se connecter ;
3. créer des sujets de vote ;
4. voter ;
5. consulter les résultats ;
6. recevoir des notifications.

L'authentification entre les services repose sur des **JWT (JSON Web Tokens)**.

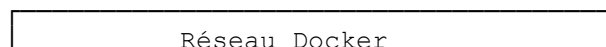
Microservices

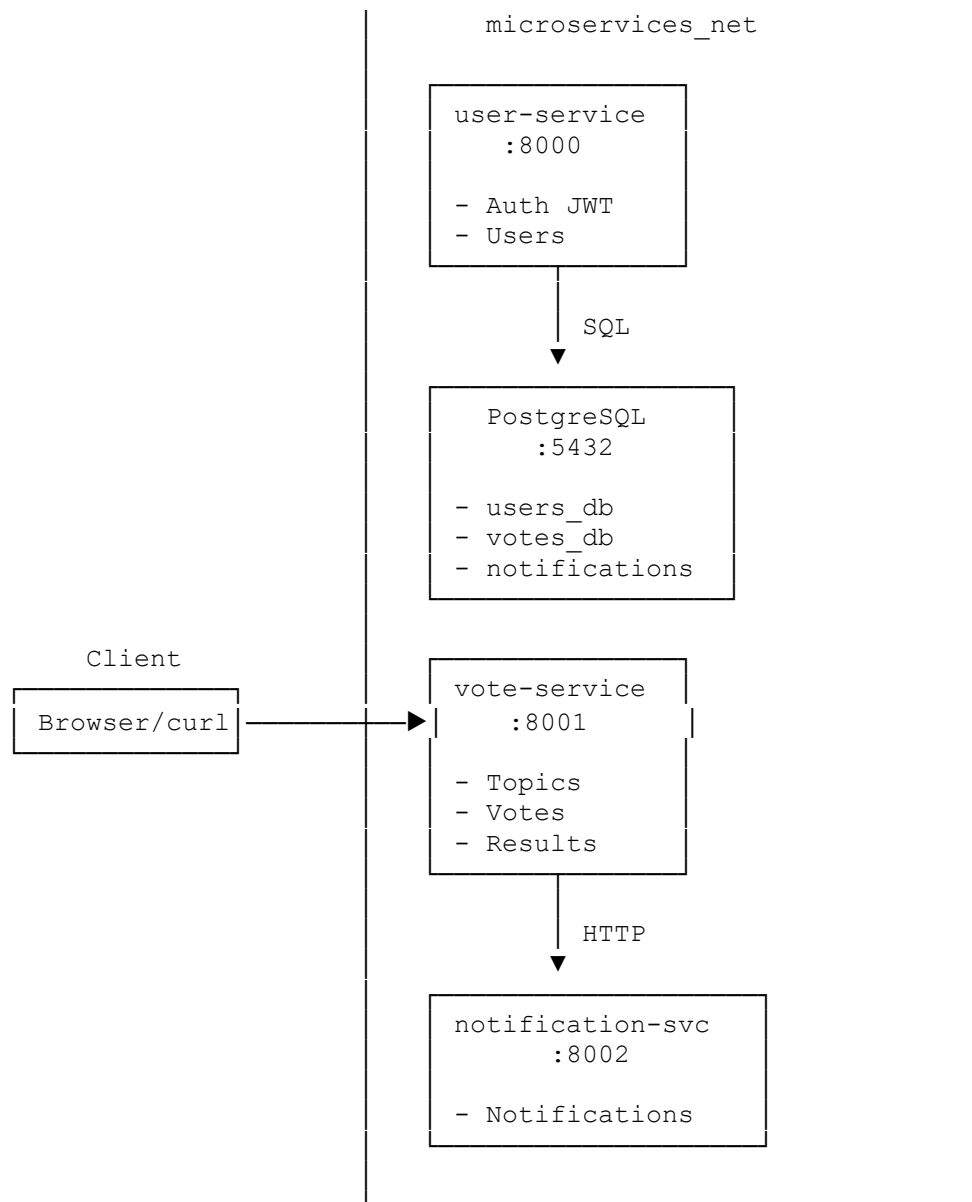
Le système est composé de trois services principaux :

Service	Responsabilité	Port
user-service	Utilisateurs et authentification JWT	8000
vote-service	Gestion des sujets et des votes	8001
notification-service	Gestion des notifications	8002

Architecture

L'architecture générale du système est la suivante :





Communication entre les services

Les services communiquent principalement avec :

1. **HTTP/REST** pour les communications inter-services ;
2. **JWT** pour l'authentification ;
3. **PostgreSQL** pour la persistance des données ;
4. **Docker Network** pour permettre aux conteneurs de communiquer entre eux.

Technologies utilisées

Technologie	Version	Rôle
Python	3.11	Langage principal
Django	4.x	Framework web
Django REST Framework	3.x	Création des API REST
SimpleJWT	5.x	Authentification JWT
PostgreSQL	15	Base de données relationnelle
Docker	≥ 20.10	Conteneurisation
Docker Compose	$\geq 2.x$	Orchestration locale
Kubernetes	1.26+	Orchestration en production
pgAdmin 4	Latest	Administration PostgreSQL

Prérequis

Avant de commencer, assurez-vous d'avoir installé :

1. **Docker** ≥ 20.10
2. **Docker Compose** $\geq 2.x$
3. **Git**
4. **kubectl** (*optionnel, pour Kubernetes*)
5. **Minikube** (*optionnel, pour tester Kubernetes localement*)

Vérifier les versions :

```
docker --version
docker compose version
git --version
```

Démarrage rapide

1. Cloner le projet

```
git clone <url-du-repo>
cd microservices_demo
```

2. Construire les images et démarrer les services

```
docker compose up --build
```

Cette commande :

1. construit les images Docker ;
2. crée les conteneurs ;
3. démarre les trois microservices ;

4. démarre PostgreSQL ;
5. connecte les services au réseau Docker.

Attendez que les services soient démarrés et indiqués comme `healthy` ou `started` dans les logs.

3. Exécuter les migrations

Les migrations doivent être exécutées dans chaque service.

user-service

```
docker compose exec user-service python manage.py migrate
```

vote-service

```
docker compose exec vote-service python manage.py migrate
```

notification-service

```
docker compose exec notification-service python manage.py migrate
```

Créer un superutilisateur

Pour créer un compte administrateur :

```
docker compose exec user-service python manage.py createsuperuser
```

Suivez ensuite les instructions affichées dans le terminal.

Services disponibles

Une fois le projet démarré :

Service	URL
User Service	http://localhost:8000
Vote Service	http://localhost:8001
Notification Service	http://localhost:8002
pgAdmin	http://localhost:5050

PI et endpoints

User Service

URL de base :

http://localhost:8000

Méthode	Endpoint	Description	Authentification
POST	/api/users/register/	Créer un compte	Aucune
POST	/api/users/login/	Obtenir les tokens JWT	Aucune
POST	/api/users/token/refresh/	Rafraîchir le token	Refresh token
GET	/api/users/me/	Profil de l'utilisateur connecté	Bearer token
GET	/api/users/list/	Liste des utilisateurs	Admin
GET	/api/users/<id>/	Détails d'un utilisateur	Bearer token
PUT	/api/users/<id>/	Modifier un utilisateur	Owner/Admin

Vote Service

URL de base :

http://localhost:8001

Méthode	Endpoint	Description	Authentification
GET	/api/votes/topics/	Liste des sujets	Optionnelle
POST	/api/votes/topics/	Créer un sujet	Admin
GET	/api/votes/topics/<id>/	Détails d'un sujet	Optionnelle
PUT	/api/votes/topics/<id>/	Modifier un sujet	Admin
DELETE	/api/votes/topics/<id>/	Supprimer un sujet	Admin
POST	/api/votes/topics/<id>/vote/	Voter	Bearer token
GET	/api/votes/topics/<id>/results/	Consulter les résultats	Optionnelle

Notification Service

URL de base :

http://localhost:8002

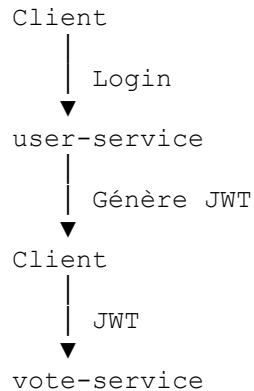
Méthode	Endpoint	Description	Authentification
GET	/api/notifications/?user_id=<id>	Liste des notifications	Optionnelle
POST	/api/notifications/create/	Créer une notification	Service-to-service
PUT	/api/notifications/<id>/read/	Marquer comme lue	Bearer token

Communication entre les microservices

Authentication JWT

L'authentification utilise des **JSON Web Tokens (JWT)**.

Lorsqu'un utilisateur se connecte :



La validation du JWT est effectuée **localement dans chaque service**.

Il n'est donc pas nécessaire pour `vote-service` ou `notification-service` de contacter `user-service` à chaque requête.

Pourquoi ?

Cela permet :

1. de réduire les appels réseau ;
2. de réduire la latence ;
3. d'améliorer la résilience ;
4. d'éviter une dépendance permanente à `user-service`.

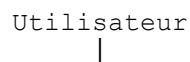
Les services utilisent la même `SECRET_KEY` pour vérifier la signature du JWT.

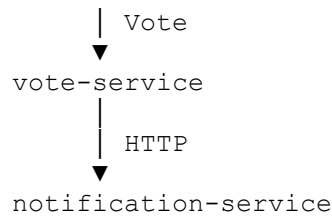
⚠ En production, la `SECRET_KEY` ne doit pas être stockée directement dans le code. Elle doit être placée dans un **Secret Kubernetes** ou un gestionnaire de secrets.

Notifications Fire-and-Forget

Lorsqu'un utilisateur vote, `vote-service` envoie une notification à `notification-service`.

Le principe est :

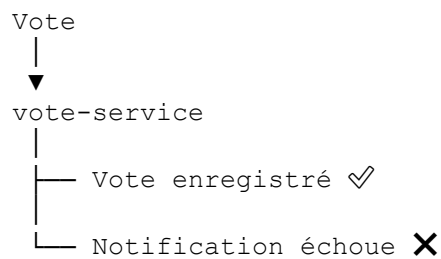




Le vote n'attend pas indéfiniment la réponse du service de notification.

Le timeout est fixé à **2 secondes**.

Si `notification-service` est indisponible :



Le vote reste donc valide.

Évolution possible

Pour une architecture plus robuste, cette communication pourrait évoluer vers une **file de messages**, par exemple :

1. RabbitMQ
2. Redis Streams

Cela permettrait notamment de mieux garantir la livraison des notifications.

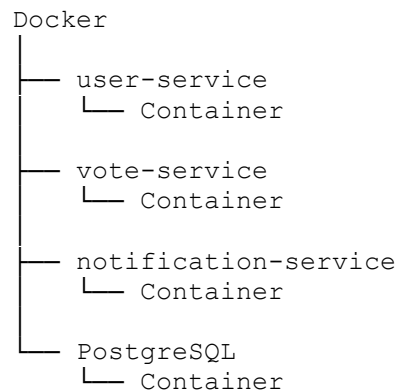
Variables d'environnement

Variable	Service	Description	Valeur Docker par défaut
SECRET_KEY	user-service	Clé Django et signature JWT	user-service-secret-key-change-in-prod
DEBUG	user-service	Mode debug Django	True
DB_NAME	user-service	Base utilisateurs	users_db
DB_NAME	vote-service	Base des votes	votes_db
DB_NAME	notification-service	Base notifications	notifications_db

USER_SERVICE_URL	vote-service	URL interne du user-service	http://user-service:8000
NOTIFICATION_SERVICE_URL	vote-service	URL interne du notification-service	http://notification-service:8002

Architecture Docker

Chaque microservice est exécuté dans son propre conteneur.



Tous les conteneurs communiquent à travers le réseau :

```
microservices_net
```

Déploiement Kubernetes

En production, les microservices peuvent être déployés avec **Kubernetes**.

Les ressources Kubernetes du projet comprennent notamment :

1. Deployments ;
2. Services ;
3. ConfigMap ;
4. Secrets ;
5. Ingress ;
6. PostgreSQL.

Pour appliquer les ressources :

```
kubectl apply -f k8s/
```

Vérifier les Pods

```
kubectl get pods
```

Vérifier les Services

```
kubectl get services
```

Vérifier l'Ingress

```
kubectl get ingress
```

Scaling

Chaque microservice peut être mis à l'échelle indépendamment.

Par exemple, pour exécuter trois instances de `vote-service` :

```
kubectl scale deployment vote-service --replicas=3
```

On obtient :

```
Kubernetes
├── vote-service Pod 1
├── vote-service Pod 2
└── vote-service Pod 3
```

Cela permet de supporter davantage de requêtes sans devoir multiplier tous les autres services.

Comprendre le chemin vers la production

Le projet suit le principe :

```
APPLICATION
  ↓
MICROSERVICE
  ↓
IMAGE DOCKER
  ↓
CONTENEUR
  ↓
POD
  ↓
KUBERNETES
  ↓
PRODUCTION
```

En termes simples

Application → le logiciel développé.

Microservice → une partie indépendante de l'application.

Image → le paquet contenant le microservice et ses dépendances.

Conteneur → l'image en cours d'exécution.

Pod → l'environnement Kubernetes qui contient le conteneur.

Kubernetes → gère et orchestre les Pods.

Production → l'environnement réel utilisé par les utilisateurs.

Commandes utiles

Démarrer le projet

```
docker compose up --build
```

Arrêter le projet

```
docker compose down
```

Voir les conteneurs

```
docker compose ps
```

Voir les logs

```
docker compose logs
```

Voir les logs d'un service spécifique :

```
docker compose logs user-service
```

Migrations

Créer les migrations :

```
docker compose exec <service-name> python manage.py makemigrations
```

Appliquer les migrations :

```
docker compose exec <service-name> python manage.py migrate
```

pgAdmin

pgAdmin est disponible avec le profil debug.

Démarrer pgAdmin :

```
docker compose --profile debug up
```

Accès :

<http://localhost:5050>

Identifiants

Email : admin@admin.com
Password : admin

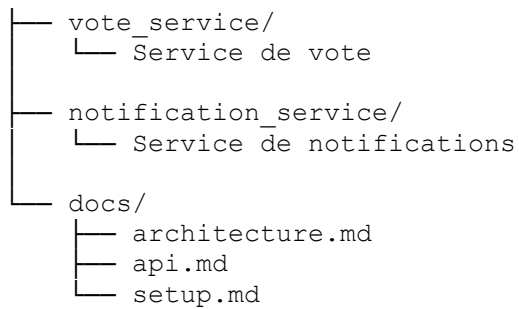
Connexion PostgreSQL

Host : db
Port : 5432
User : postgres
Password : postgres

⚠ Ces identifiants sont adaptés à un environnement de démonstration/développement. Ils doivent être remplacés par des secrets sécurisés en production.

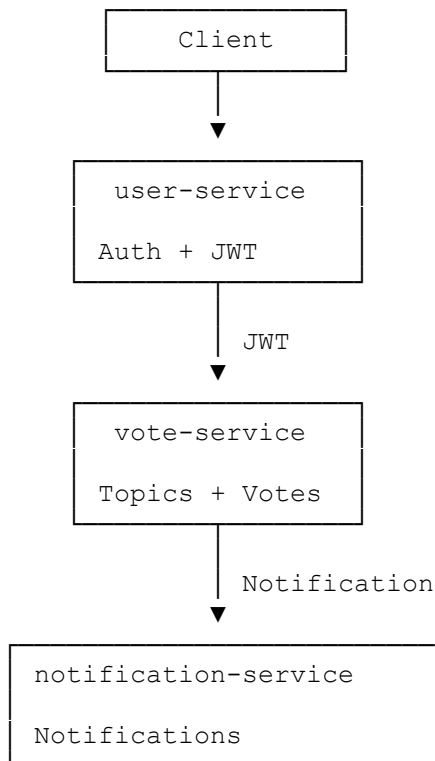
Structure du projet

```
microservices_demo/
├── docker-compose.yml
├── init-db.sh
├── k8s/
│   ├── configmap.yaml
│   ├── secrets.yaml
│   ├── postgres-deployment.yaml
│   ├── user-service-deployment.yaml
│   ├── vote-service-deployment.yaml
│   ├── notification-service-deployment.yaml
│   └── ingress.yaml
├── user_service/
│   └── Service d'authentification
```



Fonctionnement global

Le fonctionnement principal du système peut être résumé ainsi :



Ainsi, chaque service possède une **responsabilité claire**, peut être développé et déployé indépendamment et peut être mis à l'échelle selon ses propres besoins.

Objectif pédagogique

Ce projet permet de comprendre concrètement les principaux concepts d'une architecture microservices :

1. séparation des responsabilités ;

2. communication HTTP entre services ;
3. authentification JWT ;
4. conteneurisation avec Docker ;
5. orchestration locale avec Docker Compose ;
6. orchestration avec Kubernetes ;
7. scaling indépendant des services ;
8. communication asynchrone de type fire-and-forget ;
9. gestion des variables d'environnement ;
10. déploiement d'une architecture distribuée.

Projet académique

Microservices Demo — Système de vote distribué

Technologies principales :

Django REST Framework + PostgreSQL + Docker + Docker Compose + Kubernetes + JWT